

**EXERCISE 6.5**

Use `map` to reimplement `double` in a more elegant way. See exercise 6.2.

6.4.1 Combining state actions

Unfortunately, `map` isn't powerful enough to implement `intDouble` and `doubleInt` from exercise 6.3. What we need is a new combinator `map2` that can combine two RNG actions into one using a binary rather than unary function.

**EXERCISE 6.6**

Write the implementation of `map2` based on the following signature. This function takes two actions, `ra` and `rb`, and a function `f` for combining their results, and returns a new action that combines them:

```
def map2[A,B,C](ra: Rand[A], rb: Rand[B])(f: (A, B) => C): Rand[C]
```

We only have to write the `map2` combinator once, and then we can use it to combine arbitrary RNG state actions. For example, if we have an action that generates values of type `A` and an action to generate values of type `B`, then we can combine them into one action that generates pairs of both `A` and `B`:

```
def both[A,B](ra: Rand[A], rb: Rand[B]): Rand[(A,B)] =
  map2(ra, rb)((_, _))
```

We can use this to reimplement `intDouble` and `doubleInt` from exercise 6.3 more succinctly:

```
val randIntDouble: Rand[(Int, Double)] =
  both(int, double)

val randDoubleInt: Rand[(Double, Int)] =
  both(double, int)
```

**EXERCISE 6.7**

Hard: If you can combine two RNG transitions, you should be able to combine a whole list of them. Implement `sequence` for combining a `List` of transitions into a single transition. Use it to reimplement the `ints` function you wrote before. For the latter,